SOCIETYOS

Residential Society Management & Intelligent Gate Access Control

IMPLEMENTATION BLUEPRINT v1.0
Architecture • Database • API specification • User workflows • Security • Development stages • Installation • Testing

Agreed product direction

Decision	Selected direction
Product scope	Complete society management + gate access; multi-tenant SaaS
Gate types	Automatic boom barrier, sliding gate and electromagnetic door lock
Biometrics	Later phase; validate software and hardware abstraction first
First deliverable	Working software MVP with virtual gate/device simulator
Architecture	NestJS modular monolith, PostgreSQL, Next.js and local edge access layer

Prepared 30 October 2026. This document consolidates the project decisions and technical design discussed so far. It is a practical engineering blueprint, not a certified physical-security design.

Critical principle: the application determines authorization; a local access controller enforces the decision and physical safety interlocks. A cloud API response is not proof that a gate physically opened.

Contents

Section	Contents
1	Executive summary and scope
2	Technology stack and architecture
3	Repository, installation and local environment
4	Database design and initial schema
5	REST API specification
6	Resident, visitor and gate workflows
7	Gate/device integration and MQTT
8	Security, privacy and physical safety
9	Development stages and sprint plan
10	Testing, deployment and operations
11	Risks, decisions and immediate build checklist

1. Executive summary and product scope

SocietyOS is a multi-tenant residential community management platform with visitor management and physical gate access control. Residents invite guests using time-limited QR/PIN passes; guards and gate terminals verify access; an authorized local controller executes commands; and decisions and physical outcomes are recorded.

Build software first. The first release must work without biometric hardware. Use a simulated terminal and gate to validate resident onboarding, pass creation, authorization, gate commands and audit trails. Add physical hardware after the software contracts and security controls are stable.

Initial user roles

Role	Responsibilities	Boundary
Platform administrator	Onboard societies, manage platform configuration	No routine access to resident data without authorized need
Society administrator	Buildings, flats, residents, staff, gates, policies, reports	Assigned society only
Resident	Manage household, invite/revoke visitors, approve requests, view history	Only authorized flats
Security guard	Verify visitors, manage approvals, record entry/exit, report incidents	No unrestricted administration
Visitor	Present valid QR/PIN and complete required approval	Restricted kiosk interaction
Device/controller	Report health, receive commands, submit events	Authenticated; assigned gates only

First MVP features

- Multiple society tenants with strict data isolation.
- Society, building, flat, user, resident and household-member management.
- Visitor categories: relative, friend, delivery, cab, maintenance and other.
- Visitor invitations, approvals, expiry, revocation, QR/PIN credentials and history.
- Admin, resident, guard and visitor kiosk interfaces.
- Access verification engine, gate simulator, device simulator and event history.
- Device abstraction that can later support boom barriers, sliding gates, door locks and biometric readers.

Later modules

- Maintenance requests and vendor/work assignment.
- Maintenance billing, invoices, payments and expense tracking.
- Amenities, parking and booking slots.
- Announcements, polls, emergency notices and resident communication.

Do not delay the access-control MVP by implementing every society-management feature at once. Keep later modules within the same tenancy and permission framework.

2. Recommended technology stack and architecture

Layer	Technology	Purpose
Web interfaces	Next.js + TypeScript + Tailwind + shadcn/ui	Admin, resident PWA, guard and kiosk
Backend	NestJS + TypeScript	Modular monolith and business rules
Database	PostgreSQL	Relational system of record and transactions
ORM/migrations	Prisma	Typed data access and schema migrations
Authentication	OIDC provider such as Keycloak	User identity; application enforces RBAC and resource permissions
Cache/jobs	Redis + BullMQ	Caching, rate limits and background jobs
Device messaging	MQTT 5 + Mosquitto + TLS	Telemetry, commands and acknowledgements
Live updates	WebSockets	Live guard dashboard and device status
Mobile later	React Native + Expo	Native resident app after PWA validation
Notifications	Firebase Cloud Messaging + SMS provider	Push and visitor pass delivery
Storage	S3-compatible object storage	Documents/approved attachments; minimize sensitive data
Deployment	Docker + Linux + Nginx	Repeatable development and initial production
Monitoring	Prometheus + Grafana + centralized logs	API, broker and device health
Testing	Jest, Supertest, Playwright, k6	Unit, integration, browser and load tests
Hardware	Vendor SDK adapters + generic controller interface	Validated Suprema, ZKTeco or compatible controllers

Why this combination

- PostgreSQL is a better primary database than MongoDB for society/flat/resident/pass/device relationships and atomic redemption.
- NestJS provides modules, dependency injection, guards, validation and a maintainable structure for many business domains.
- Use a modular monolith initially; do not introduce microservices for a 20-flat pilot.
- Use REST/HTTPS for user/admin APIs, MQTT for device messaging and WebSockets for live UI updates.
- Use Keycloak/OIDC rather than building a custom OAuth authorization server. Authentication does not replace application-level authorization.
- Use a PWA first. Build React Native only when native capabilities or product requirements justify it.
- Use Docker Compose initially. Kubernetes is not required for the first pilot.

Logical architecture

Layer	Components
Client	Admin • Resident • Guard • Visitor kiosk
Web edge	Nginx • HTTPS/TLS • rate limits
Backend	NestJS modules: identity, tenant, society, residents, visitors, access, devices, notifications, audit
Data/jobs	PostgreSQL source of truth • Redis/cache/queues • object storage
Device messaging	MQTT broker • per-device identity • topic ACLs • TLS
Society edge	Local controller • bounded offline policy • local event queue • safety interlocks
Physical	QR/PIN terminal • biometric reader later • relay/controller • sensors • gate motor/lock

Architecture boundaries

- Business services own resident, flat, visitor and policy rules.
- Access Control Engine evaluates credential, society, host, gate, time, approval, revocation and use limit.
- Device service owns provisioning, device identity, health, commands and events.
- Local controller enforces approved offline rules and physical safety interlocks.
- Gate adapters translate normalized commands into vendor-specific protocols.
- Record authorization, command acknowledgement, gate movement and entry confirmation separately.

3. Repository and installation

Suggested monorepo

```
society-os/
  apps/
    web/                # Admin, resident and guard route groups initially
    api/                # NestJS backend
    device-simulator/   # Virtual controller and gate
  packages/
    shared-types/
    validation/
    api-client/
    ui/
    config/
  infrastructure/
    docker/
    mqtt/
    postgres/
    monitoring/
  docs/
    architecture/
    database/
    api/
    security/
    hardware/
  docker-compose.yml
  pnpm-workspace.yaml
  turbo.json
  README.md
```

Required tools

Tool	Why needed
Git + GitHub	Version control and collaboration
Node.js supported LTS + pnpm	TypeScript runtime/package management
Docker Desktop	Local PostgreSQL, Redis and MQTT
VS Code	Development environment
Postman or Bruno	API testing
DBeaver	Inspect PostgreSQL
Prisma CLI	Schema and migrations
Playwright	Browser E2E tests
MQTT Explorer	Inspect device topics/messages

Initial setup commands

```
mkdir society-os
cd society-os
git init
mkdir apps packages infrastructure docs

# Create workspace file
cat > pnpm-workspace.yaml <<'EOF'
packages:
  - "apps/*"
  - "packages/*"
EOF

pnpm add -Dw turbo typescript

# Create web and API
pnpm create next-app apps/web
pnpm dlx @nestjs/cli new apps/api
```

Backend initial dependencies

```
cd apps/api
pnpm add @nestjs/config @nestjs/swagger
pnpm add class-validator class-transformer
pnpm add @prisma/client
pnpm add ioredis bullmq mqtt
pnpm add -D prisma
```

Prisma setup

```
npx prisma init
# Configure DATABASE_URL in .env
npx prisma migrate dev --name initial_schema
npx prisma generate
```

Example local environment variables

```
DATABASE_URL=
REDIS_URL=
MQTT_URL=
OIDC_ISSUER_URL=
OIDC_CLIENT_ID=
OIDC_CLIENT_SECRET=
CREDENTIAL_HMAC_KEY=
APP_BASE_URL=
```

Local infrastructure

Create Docker Compose services for PostgreSQL, Redis and Mosquitto. Bind development ports to localhost. Disable anonymous MQTT access. Production requires TLS, per-device credentials, topic ACLs, secret management, private networking, durable storage and restricted exposure. Never expose PostgreSQL, Redis or broker administration to the public internet.

Version note

The earlier discussion used Node.js 24, PostgreSQL 17 and Prisma 6 as an example baseline. Before setup, verify current support and compatibility, then pin tested versions in the lockfile. Do not blindly upgrade major versions mid-sprint.

4. Database design

Use UUID primary keys and TIMESTAMPTZ timestamps. Store time in UTC and render using the society timezone. Every society-owned row must have society_id directly or an enforceable parent relationship.

Domain	Tables	Purpose
Identity	users, roles, permissions, role_permissions, sessions	Identity, role and permissions
Tenancy	societies, society_memberships	Tenant boundaries and user membership
Residential	buildings, flats, resident_flat_memberships	Buildings, flats and multiple resident-to-flat links
Visitors	visitors, visitor_passes, visitor_pass_credentials, visitor_pass_access_points, visitor_approvals, visitor_pass_uses	Guest identity, authorization, credentials, approval and redemption
Devices	access_points, devices, device_credentials, device_access_points, gate_commands, device_events	Physical access points and connected devices
Security	access_events, audit_logs, security_alerts, device_health_logs	Access decisions, admin changes and operational health
Later	maintenance_requests, invoices, payments, amenities, bookings, announcements, notifications	Full society management modules

Key relationships

- Society has many buildings, flats, memberships, access points and devices.
- Building has many flats; flat number is unique within a building.
- User can belong to multiple societies and multiple flats.
- Society membership role is separate from flat relationship (owner, tenant, family member).
- Visitor pass belongs to one society, host, flat and visitor; it authorizes explicitly selected access points.
- Device can be associated with one or more access points, according to the physical topology.
- Access events are tenant-scoped and linked to source device/access point.

Important constraints and indexes

```
CREATE UNIQUE INDEX uq_flat_building_number
ON flats (building_id, flat_number);

CREATE UNIQUE INDEX uq_society_user
ON society_memberships (society_id, user_id);

CREATE UNIQUE INDEX uq_device_uid
ON devices (device_uid);

CREATE UNIQUE INDEX uq_device_access_point
ON device_access_points (device_id, access_point_id);

CREATE UNIQUE INDEX uq_redemption_id
ON visitor_pass_uses (redemption_id);
```

Tenant isolation

- Enforce tenant authorization in backend services for every read and write; never rely on frontend filtering.
- Use composite foreign keys to ensure a pass, flat, device and access point belong to the same society.
- Use PostgreSQL Row-Level Security as defense in depth; set tenant context transactionally and test connection-pool behavior.
- Test nested-resource and join paths for cross-tenant data leakage.
- Use database least privilege. Runtime application role must not own tables or bypass RLS.

Visitor pass lifecycle

```
DRAFT
  -> PENDING_APPROVAL (when policy requires)
  -> ACTIVE
       -> REVOKED
       -> EXPIRED
       -> EXHAUSTED
```

Prisma starter schema

The following is a compact conceptual starter. Add composite foreign keys, redemption, approval, audit and device-command models before production migrations. Prisma relation syntax and generated client configuration must match the pinned Prisma major version.

```
enum PassStatus {
  DRAFT
  PENDING_APPROVAL
  ACTIVE
  REVOKED
  EXPIRED
  EXHAUSTED
}

model User {
  id           String @id @default(uuid()) @db.Uuid
  name         String
  email        String? @unique
  phone        String? @unique
  passwordHash String?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  memberships SocietyMembership[]
}

model Society {
  id          String @id @default(uuid()) @db.Uuid
  name        String
  slug        String @unique
  timezone    String @default("Asia/Kolkata")
  memberships SocietyMembership[]
  buildings   Building[]
}

model SocietyMembership {
  id        String @id @default(uuid()) @db.Uuid
  societyId String @db.Uuid
  userId    String @db.Uuid
  role      String
  status    String
  society   Society @relation(fields: [societyId], references: [id])
  user      User @relation(fields: [userId], references: [id])
  @@unique([societyId, userId])
}

model Building {
  id        String @id @default(uuid()) @db.Uuid
  societyId String @db.Uuid
  name      String
  society   Society @relation(fields: [societyId], references: [id])
  flats     Flat[]
}

model Flat {
  id         String @id @default(uuid()) @db.Uuid
  buildingId String @db.Uuid
  flatNumber String
  building   Building @relation(fields: [buildingId], references: [id])
  @@unique([buildingId, flatNumber])
}

model VisitorPass {
  id         String @id @default(uuid()) @db.Uuid
  societyId  String @db.Uuid
  flatId     String @db.Uuid
  hostUserId String @db.Uuid
  visitorId  String @db.Uuid
  status     PassStatus @default(DRAFT)
  validFrom  DateTime
  validUntil DateTime
  maxUses    Int @default(1)
  usedCount  Int @default(0)
  createdAt  DateTime @default(now())
}
```

Security event history must be protected from alteration. Do not delete history when a resident or visitor account is disabled. Apply lawful retention and anonymization rules to personal data.

5. API specification

Base path: /api/v1. Use JSON, UUIDs, ISO 8601 timestamps, consistent error envelopes and cursor pagination. Require authentication and resource-level authorization unless a route is explicitly designed for a restricted kiosk/device flow.

Area	Method	Endpoint	Purpose
Auth	POST	/auth/login	OIDC/session login
Auth	POST	/auth/refresh	Rotate refresh/session token
Auth	POST	/auth/logout	Revoke session
Auth	GET	/auth/me	Current user and memberships
Society	POST	/societies	Controlled society onboarding
Society	GET	/societies/:id	Society details
Society	PATCH	/societies/:id	Update society
Building	POST	/societies/:id/buildings	Create building
Flat	POST	/buildings/:id/flats	Create flat
Flat	GET	/societies/:id/flats	List authorized flats
Resident	POST	/societies/:id/residents	Invite/register resident
Resident	POST	/flats/:id/members	Add household member
Resident	DELETE	/flats/:id/members/:userId	End membership
Visitor	POST	/visitor-passes	Create visitor pass
Visitor	GET	/visitor-passes	List authorized passes
Visitor	POST	/visitor-passes/:id/approve	Approve pass
Visitor	POST	/visitor-passes/:id/revoke	Revoke pass
Device	POST	/societies/:id/access-points	Create gate/access point
Device	POST	/societies/:id/devices	Provision device
Device	GET	/devices/:id/health	Device health
Access	POST	/access/verify	Verify credential via authenticated device

Access verification example

```
POST /api/v1/access/verify
Authorization: authenticated device identity
Idempotency-Key: <unique-request-key>

{
  "deviceId": "<device-uuid>",
  "accessPointId": "<gate-uuid>",
  "credentialType": "PIN",
  "credential": "<visitor-secret>",
  "requestId": "<unique-uuid>"
}

Success:
{
  "decision": "AUTHORIZED",
  "reasonCode": "VALID_VISITOR_PASS",
  "accessEventId": "<event-uuid>",
  "commandId": "<command-uuid>",
  "expiresAt": "<short-command-expiry>"
}

Denial:
{
  "decision": "DENIED",
  "reasonCode": "PASS_EXPIRED",
  "accessEventId": "<event-uuid>"
}
```

Never trust kiosk-supplied society ID, resident ID or role. Derive device identity and tenant/access-point scope from authenticated device records. Do not log PINs, QR tokens, passwords or access tokens.

Reason codes

```
VALID_VISITOR_PASS
PASS_EXPIRED
PASS_REVOKED
PASS_ALREADY_USED
INVALID_CREDENTIAL
WRONG_ACCESS_POINT
WRONG_SOCIETY
APPROVAL_REQUIRED
DEVICE_DISABLED
RATE_LIMIT_EXCEEDED
CONTROLLER_UNAVAILABLE
SAFETY_INTERLOCK_ACTIVE
INTERNAL_ERROR
```

HTTP conventions

- 400 malformed request; 401 unauthenticated; 403 unauthorized; 404 not found; 409 state/idempotency conflict; 422 business validation failure; 429 rate limit; 5xx server failure.
- Use a consistent error response with code, safe message and requestId.
- Use idempotency keys for pass creation and retryable mutations.
- Document schemas in OpenAPI and generate/maintain a typed frontend API client.

6. Core product workflows

A. Society onboarding

1. Platform operator creates a society tenant through controlled onboarding.
2. Society profile, timezone, policy defaults and administrator membership are created.
3. Society admin creates buildings and flats.
4. Admin invites residents and assigns owner/tenant/family relationships.
5. Residents verify their accounts and accept invitations.
6. Admin creates access points and registers simulated devices.

B. Resident invites a relative

1. Resident signs in and selects an authorized flat.
2. Resident enters visitor name, optional phone, category, expected arrival window, allowed gate and maximum uses.
3. Backend checks resident membership, flat relationship, society status, gate membership, duration policy and invitation limits.
4. Backend generates a random QR token and/or PIN; stores only protected verification material.
5. If required, pass enters PENDING_APPROVAL; otherwise it becomes ACTIVE.
6. Notification delivers visitor instructions. Resident can view and revoke the pass.
7. Pass expires at valid_until or becomes EXHAUSTED after its permitted uses.

Example invitation payload

```
POST /api/v1/visitor-passes
Idempotency-Key: <unique-key>

{
  "flatId": "<flat-uuid>",
  "visitor": {
    "name": "Rahul Sharma",
    "phone": "+919876543210"
  },
  "category": "RELATIVE",
  "validFrom": "2026-10-30T17:00:00+05:30",
  "validUntil": "2026-10-30T21:00:00+05:30",
  "maxUses": 1,
  "accessPointIds": ["<main-gate-uuid>"]
}
```

C. Visitor arrives

1. Visitor scans QR or enters PIN at kiosk.
2. Authenticated terminal sends credential and access-point context.
3. Access engine checks device, pass status, society, host/flat, gate, time, approval, revocation and use limit.
4. Database transaction atomically consumes the permitted use and creates redemption/access records.
5. Short-lived authenticated command is sent to the local controller.
6. Controller checks command expiry, replay protection and physical safety interlocks.
7. Controller acknowledges execution; sensors report actual gate state.
8. System records decision, command result, gate state and sends notification.

D. Delivery visitor

Society policy can require a resident-generated delivery pass or resident/guard approval after arrival. Delivery workers must not receive permanent shared credentials. Use category-specific validity windows, gate scope, attempt limits and audit records.

E. Resident biometric entry (later)

1. Administrator provisions an exact compatible biometric reader/controller.
2. Resident enrollment uses the approved vendor workflow and privacy notice/process.
3. Prefer local matching on the reader; central platform stores minimum enrollment reference/status only.
4. Device reports authenticated access event; membership/policy is checked according to the device architecture.
5. Controller operates according to certified safety behavior and reports sensor-confirmed state.

F. Internet outage

1. Local controller detects cloud/broker loss and raises an operational alert.
2. Resident credentials continue only if securely cached and permitted by offline policy.
3. Pre-issued visitor passes may work offline only if synchronized, unexpired and allowed by bounded risk policy.
4. New invitations and immediate revocations may not reach the controller while disconnected.
5. Controller buffers signed/sequence-numbered events and synchronizes on recovery.
6. Stale commands and out-of-policy credentials are rejected. Emergency egress/fire release remains independent.

7. Gate and device integration

Support boom barriers, sliding gates and electromagnetic door locks through a normalized access-point model and adapter pattern. The visitor workflow should not be rewritten for each hardware type.

Gate family	Control mechanism	Safety concerns
Boom barrier	Barrier controller input/relay and sensors	Vehicle detection, obstruction protection, anti-tailgating and emergency operation
Sliding gate	Motor controller open/close/stop inputs and limit sensors	Obstruction detection, travel limits, manual release and safe movement
Electromagnetic lock	Access controller lock output and door contact	Fire release, emergency egress, fail-safe/fail-secure behavior and door-held-open detection

Normalized gate interface

```
type GateState =
  | "CLOSED" | "OPENING" | "OPEN" | "CLOSING"
  | "STOPPED" | "FAULT" | "UNKNOWN";

type GateCommand = "OPEN" | "CLOSE" | "STOP";

interface GateController {
  open(): Promise<void>;
  close(): Promise<void>;
  stop(): Promise<void>;
  getState(): Promise<GateState>;
  getHealth(): Promise<boolean>;
}
```

Adapters

```
GateController
  ■■■ SimulatedGateAdapter
  ■■■ BoomBarrierAdapter
  ■■■ SlidingGateAdapter
  ■■■ DoorLockAdapter
  ■■■ SupremaAdapter     # exact model/SDK validation required
  ■■■ ZKTecoAdapter      # exact model/SDK validation required
```

MQTT topic convention

```
society/{societyId}/device/{deviceId}/telemetry
society/{societyId}/device/{deviceId}/health
society/{societyId}/device/{deviceId}/events
society/{societyId}/device/{deviceId}/commands
society/{societyId}/device/{deviceId}/acknowledgements
```

Every command must include unique command ID, target device/access point, command type, issue time, short expiry, nonce and authenticated integrity. Devices must have individual credentials and topic ACLs. Do not retain stale gate-open commands. Monitor device clock drift.

Hardware procurement checklist

- Exact model, firmware, SDK/API documentation, licensing and vendor support confirmed.
- Local LAN operation and offline authorization verified.
- Relay/controller interface and motor compatibility reviewed by installer.
- Sensor feedback and command acknowledgement available.
- Device identity, secure communication and credential rotation supported.
- Secure firmware updates and vulnerability support documented.
- Biometric matching location, template storage, liveness/PAD and non-biometric alternative documented.
- Emergency release, fire integration, manual override and obstruction detection reviewed by qualified professional.

Do not buy hardware based only on a product family name or marketing claim. Verify exact model, firmware, SDK, licensing and integration behavior on a lab bench before purchase/installation.

8. Security, privacy and physical safety

Area	Required control
User authentication	OIDC; secure sessions/refresh; MFA for privileged administrators where appropriate
Authorization	RBAC plus society, flat, pass, gate and resource-level checks on every request
Tenant isolation	Backend enforcement, composite constraints and PostgreSQL RLS defense in depth
Visitor PIN	Random, limited lifetime, keyed verification hash, rate limits, attempt monitoring
QR token	High-entropy opaque or signed short-lived token; no personal data embedded
Replay/concurrency	Nonce, command expiry, unique request ID, atomic redemption and idempotency
Device identity	Individual device certificate or equivalent; rotate/revoke credentials
Transport	TLS for API/MQTT; broker ACLs; restricted networks
Data at rest	Encrypted disks/backups, least privilege, minimized sensitive fields
Audit	Append-only security records, restricted permissions, protected backup
Monitoring	Failed attempts, device disconnects, reboot, command timeout and unusual access
Recovery	Encrypted backup, tested restore, incident runbook and recovery objectives

Visitor PIN and QR rules

- Six-digit PINs have limited entropy; do not treat them as strong authentication alone.
- Apply per-device/account/credential rate limits, backoff, alerts and attempt limits.
- Store PIN verification material using keyed HMAC with a protected versioned key; never log plaintext.
- Use high-entropy opaque QR tokens or short-lived signed credentials.
- Bind each pass to society, flat, host, allowed gate, validity window and use limit.
- Use transaction locking/conditional updates and unique redemption IDs to prevent concurrent reuse.

Biometric privacy

Prefer on-device matching and protected local templates. Avoid central raw face/fingerprint images. A normal cryptographic hash of a biometric template is not a practical replacement for a biometric matching system. Define purpose, lawful basis, privacy notices, alternatives, retention, deletion, access controls and vendor processing terms.

India legal review

Review the Digital Personal Data Protection Act, 2023 and DPDP Rules, 2025, including phased commencement and the obligations applicable on the actual deployment date. Obtain qualified legal advice on resident/visitor data, biometrics, CCTV if added, notices, retention, grievance handling and vendor contracts. Do not assume GDPR alone is the applicable framework.

Physical safety — mandatory

- Emergency egress and fire alarm release must not depend on cloud, app or MQTT availability.
- Motor obstruction detection, limit switches and safe stop behavior must be implemented by appropriate hardware.
- Define fail-safe/fail-secure behavior separately for doors, boom barriers and sliding gates.
- Manual release and emergency override must be documented and accessible to authorized staff.
- Authorization is not proof of gate-open state; use physical sensors.
- Qualified access-control/gate professionals must review wiring, installation and safety behavior.

Useful security references

- OWASP API Security Top 10 and OWASP ASVS.
- NIST digital identity guidance as a reference for authentication and biometric safeguards.
- ONVIF access-control profiles where relevant; verify actual device conformance and supported features.

9. Development stages and sprint roadmap

Stage 1 — Foundation and tenant structure

Goal: administrator creates a society, buildings, flats and resident memberships.

Workstream	Tasks	Deliverable
Repository	Git, workspace, strict TypeScript, lint/format, CI	Repeatable project
Infrastructure	Docker PostgreSQL/Redis; broker later	Local services
Backend	NestJS, config, Prisma, validation, Swagger, health	Running API
Identity	OIDC/auth flow, session handling	Admin login
Tenant model	Societies, memberships, roles	Tenant boundary
Residential	Buildings, flats, resident links	20-flat apartment
Security	Tenant guards, constraints and negative tests	Isolation proven

Exit: two test societies exist; each admin manages only its own society; resident and flat creation works; cross-tenant tests pass.

Stage 2 — Resident and visitor management

- Household membership and resident verification.
- Visitor categories, invitations, approval, expiry and revocation.
- PIN/QR generation and protected credential storage.
- Pass scope: society, flat, host, gate, validity and use limit.
- Notifications via transactional outbox and retryable worker.
- Atomic redemption and rate limiting.

Exit: valid, expired, revoked, wrong-gate, wrong-society, replay and concurrent redemption cases produce correct results.

Stage 3 — Guard dashboard and virtual kiosk

- Guard login and restricted role.
- Visitor search and pending approval queue.
- PIN entry and QR scan.
- Clear allow/deny response without exposing resident private details.
- Manual approval only under policy, with reason and audit.
- Entry/exit records and arrival notification.

Exit: complete invitation-to-verification journey works without physical hardware.

Stage 4 — Access engine and gate simulator

- Gate state machine: CLOSED, OPENING, OPEN, CLOSING, STOPPED, FAULT, UNKNOWN.
- Simulated boom barrier, sliding gate and door-lock adapters.
- Command IDs, expiry, acknowledgement and sensor state simulation.
- MQTT integration with test TLS/ACLs.
- Device provisioning, heartbeat and event ingestion.
- Simulate broker loss, device offline, stale command and sensor fault.

Exit: no unauthorized simulated gate opens; state confirmation is separate from authorization.

Stage 5 — Offline edge controller

- Local controller and protected local storage.
- Secure synchronization of policies, resident credentials and approved visitor passes.
- Bounded offline validity and revocation staleness policy.
- Local event queue, sequence numbers and synchronization.
- Clock sync, stale policy and tamper/restart handling.
- Emergency egress independent of cloud.

Exit: offline behavior tested and clearly communicated.

Stage 6 — Mobile and society modules

- React Native app if PWA validation justifies it.
- Push notifications and deep links.
- Maintenance, announcements, billing, payments and amenities.
- Subscription and tenant lifecycle management.

Stage 7 — Physical hardware pilot

- Select exact model and obtain SDK, wiring and license documentation.
- Bench-test one device before a live gate installation.
- Implement vendor adapter; validate device identity, relay and sensor feedback.
- Test power/network failure, manual release, fire/emergency and obstruction.
- Pilot a controlled entrance before wider rollout.

Stage 8 — Production hardening

- Independent API/application penetration test and device security review.
- Load, concurrency, failover, backup/restore and disaster recovery tests.
- Privacy/legal review.
- Guard training, manuals, incident runbooks and resident onboarding.
- Phased rollout, monitoring and rollback plan.

First sprint — 10 working days

Day	Work	Expected result
1	Repo/workspace, Next.js, NestJS, CI baseline	Project builds
2	Docker, PostgreSQL, Prisma, migration	Database runs
3	OIDC/auth and session handling	Admin login
4	Roles, permissions, tenant context	Protected APIs
5	Society creation/settings	Society onboarding
6	Buildings and flats	Residential structure
7	Resident registration and flat membership	Residents linked
8	Tenant isolation, constraints and negative tests	Cross-tenant tests pass
9	Swagger, API tests, error envelope, audit baseline	Documented API
10	Regression, seed data, README and demo	Sprint 1 release

Sprint 1 definition of done

- Clean-machine setup documented and works.
- Migrations and seed scripts run.
- Admin can log in and create society, buildings and flats.
- Residents can be registered and linked to flats.
- Society A cannot read or mutate Society B data.
- API validation and errors are consistent; tests pass in CI.

10. Testing, deployment and operations

Development seed data

Entity	Suggested test quantity
Societies	2
Buildings	2
Flats	20 total or 10 per society
Society admins	2
Residents	20+ synthetic test users
Security guards	2
Access points	1–2 per society
Simulated devices	1 per access point
Visitor passes	Valid, expired, revoked, used, wrong-gate and replay cases

Testing matrix

Scenario	Expected result
Valid pass at permitted gate/time	Authorized; command and sensor state separately recorded
Expired/revoked pass	Denied; no gate command
Single-use pass reused	Denied
Wrong society/gate	Denied; no cross-tenant leak
Concurrent redemption	At most one permitted use
Resident removed from flat	Cannot create/manage passes for that flat
Device disabled/certificate revoked	Device operation rejected
Broker unavailable	No unauthorized opening; offline policy applies
Internet unavailable	Only bounded offline policy applies
Obstruction/fault sensor	Controller does not perform unsafe movement
Emergency/fire condition	Independent certified emergency behavior works
Database unavailable	No unsafe cloud authorization; edge policy applies

Test tools

- Jest — unit tests.
- Supertest — API integration tests.
- Playwright — admin/resident/guard/kiosk workflows.
- k6 — load and concurrency tests.
- MQTT test clients and broker/device failure simulation.
- Independent penetration test before production.

Deployment topology

Component	Initial deployment	Production requirements
Web/API	Docker containers behind Nginx	TLS, health checks, secure headers, rate limits
PostgreSQL	Managed DB preferred	PITR, private network, least privilege, restore test
Redis	Managed/protected instance	Authentication, private network, persistence as needed
MQTT	Restricted broker	TLS, device identities, topic ACLs, no anonymous access
Edge controller	On-prem Linux/embedded device	Secure storage, watchdog, tamper controls, offline policy
Monitoring	Prometheus/Grafana/logging	Alerts, retention, access controls, runbooks
Backups	Automated encrypted backups	Restore tests and defined recovery objectives

Monitor

- API availability, latency, errors and authentication failures.
- Visitor allow/deny counts and reason codes.
- Device last-seen, connection, firmware and clock drift.
- MQTT command/ack latency and timeout rates.
- Gate command result versus sensor-confirmed state.
- Offline duration, event queue depth and sync lag.
- Repeated PIN failures and unusual access patterns.

Runbooks before pilot

- Device disconnected / broker unavailable.
- Cloud/API outage and offline mode.
- Gate stuck open/closed or sensor fault.
- Lost/stolen device credential and certificate revocation.
- Suspected visitor credential compromise.
- Resident moves out or flat membership ends.
- Database restore, rollback and incident response.
- Fire/emergency operation and manual gate procedure.

11. Risks, decisions and immediate build checklist

Risk	Mitigation
Cross-society data exposure	Backend tenant checks, composite constraints, RLS and negative tests
PIN guessing/replay	Random credentials, attempt limits, expiry, atomic redemption and monitoring
Cloud/network outage	Local controller, bounded offline policy and local event queue
Vendor SDK incompatibility	Validate exact model/firmware/SDK before procurement; adapter layer
Unsafe gate movement	Certified controller, obstruction sensors, emergency release and qualified installer
Biometric privacy	Prefer on-device matching; minimize collection; legal/privacy review
Lost notification	Transactional outbox and retryable worker
Duplicate/stale gate command	Idempotency, command ID, expiry, nonce and no retained command
Early overengineering	Modular monolith, Docker Compose and staged scope

Decisions to confirm before hardware deployment

- Exact gate types, number of entrances and traffic direction.
- Dedicated kiosk versus guard-operated PIN/QR terminal.
- Which visitor categories require resident approval.
- Pass duration, use limits and allowed access windows.
- Maximum offline period, resident cache and visitor offline policy.
- Exact biometric/controller model, firmware, SDK and licensing.
- Edge controller location, physical protection and power backup.
- Emergency release/fire integration approved by qualified professionals.
- Privacy notices, lawful basis, retention and vendor data-processing terms.
- Hosting region, support owner, incident response and SaaS pricing model.

Immediate build checklist

1. Freeze stack: Next.js, NestJS, PostgreSQL, Prisma, OIDC, Redis and Docker.
2. Create Git repository and pnpm workspace.
3. Start PostgreSQL and Redis locally.
4. Create initial schema for users, societies, memberships, buildings and flats.
5. Implement authentication and tenant context.
6. Build society/building/flat/resident admin screens.
7. Create two-society seed data and prove isolation.
8. Complete Sprint 1 acceptance tests before starting visitor passes.

First demo milestone

An administrator logs in, creates Society A, creates a building with 20 flats, registers residents and assigns them to flats. A second administrator manages Society B but cannot view Society A. This proves the SaaS foundation before visitor access is added.

Final recommendation

Build a multi-tenant modular monolith using Next.js, NestJS and PostgreSQL. Use OIDC for people, device certificates for controllers, MQTT for device messaging and a local edge controller for bounded offline access. Start with QR/PIN visitor workflows and a virtual gate. Integrate biometrics and physical gate hardware only after tenant isolation, access-control logic, audit history and simulation have passed testing.

Official technical references

Technology / standard	Reference
NestJS	https://docs.nestjs.com/
Next.js	https://nextjs.org/docs
PostgreSQL	https://www.postgresql.org/docs/
Prisma	https://www.prisma.io/docs/
Keycloak	https://www.keycloak.org/documentation
Mosquitto	https://mosquitto.org/documentation/
React Native	https://reactnative.dev/docs/getting-started
OWASP ASVS	https://owasp.org/www-project-application-security-verification-standard/
OWASP API Security	https://owasp.org/www-project-api-security/
ONVIF	https://www.onvif.org/profiles/
NIST digital identity	https://pages.nist.gov/800-63-4/

Document status: Implementation Blueprint v1.0. Update when exact gate hardware, offline policy, hosting topology or legal requirements are confirmed. The schema and API examples are starter specifications and require engineering review, migrations, security tests and physical safety validation before production.